

# TEMA 6 - ASPECTOS AVANZADOS

Francisco Gortázar



# TYPE HINTS

- Son anotaciones que indican los tipos esperados de los argumentos y valor devuelto por una función
- Proporcionan información de tipos a los desarrolladores
- El intérprete los ignora:
  - No hay compilación, por lo que no se pueden verificar en tiempo de edición
  - En ejecución son ignorados y el intérprete no verifica si se pasa un valor del tipo correcto

```
1 def suma(a: int, b: int) -> int:  
2     return a + b
```



# TYPE HINTS

- Se pueden asignar tipos a variables también

```
1 a: int = 0  
2 b: bool
```

- También se puede hacer con colecciones:

```
1 names: list[str]  
2 prices: dict[str, float] = {"milk", 0.89}
```



# TYPE ALIAS

- Permiten dar un nombre nuevo a tipos ya existentes
- Útil para hacer el código más entendible
- Asignamos el alias con la palabra reservada **type** (Python 3.12+)

```
1 type Vector = list[float]
2
3 def scalar_product(scalar: float, v: Vector) -> float:
4     return [scalar * x for x in v]
5
6 result = scalar_product(2.0, [1.0, -4.2, 5.4])
```



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Cuando recorremos una colección, nuestro código es llamado por cada elemento de la misma:

```
1 for name in names:  
2     print(name)
```

- Esto es el resultado de invocar implícitamente una función generadora o iterador



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Una función normal se ejecuta de principio a fin y usa `return` para entregar un valor y finalizar
- Las funciones generadoras devuelven varios valores, pero uno cada vez que son llamadas:
  - Entrega un valor y pausa su ejecución guardando su estado para continuar justo donde se quedó la próxima vez que se le llame
  - Evaluación perezosa: no se genera un valor hasta que no se pide



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Las funciones generadoras no pueden usar `return` para devolver un valor
  - La llamada a la función finalizaría como una función normal
- En su lugar deben usar la palabra reservada **yield** para devolver el siguiente valor



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Cuando llamas a una función generadora, no obtienes el resultado inmediatamente, sino un objeto generador
- Para obtener los valores, debes iterar sobre él (por ejemplo, con un bucle **for**).
- Ejemplo: fibonacci como función generadora

```
1 def fibonacci(n):  
2     a, b = 0, 1  
3     for _ in range(n):  
4         yield a  
5         a, b = b, a + b
```





# DEFINICIÓN DE FUNCIONES GENERADORAS

- Uso:

```
1 for num in fibonacci(10):  
2     print(num)
```



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Las funciones generadoras permiten trabajar con colecciones de elementos sin tener que guardarlos todos en memoria:
- Ejemplo con listas:

```
1 def fibonacci(num_terminos):
2     a, b = 0, 1
3     lista_num_terminos = []
4     while(num_terminos > 0):
5         lista_num_terminos.append(a)
6         a = b
7         b = a + b
8         num_terminos -= 1
9     return lista_num_terminos
10 lista = fibonacci(100)
11 suma = sum(lista) # lista num terminos contiene los num_terminos
```



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Las funciones generadoras permiten trabajar con colecciones de elementos sin tener que guardarlos todos en memoria:
- Ejemplo con función generadora:

```
1 def fibonacci(n):  
2     a, b = 0, 1  
3     for _ in range(n):  
4         yield a  
5         a, b = b, a + b  
6 suma = sum(fibonacci(100)) # Sólo se mantiene un elemento en memoria cada v
```



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Ejemplo:

```
1 it = fibonacci(10)
2 print(type(it))
```

- Produce:

```
1 <class 'generator'>
```



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Internamente Python convierte la función en una clase con el método `__next__()` que permite obtener el siguiente elemento:

```
1 it = fibonacci(10)
2 for i in range(10):
3     print(it.__next__())
```



# DEFINICIÓN DE FUNCIONES GENERADORAS

- Una vez consumidos los elementos de la función generadora, tratar de obtener más valores lanza la excepción `StopIteration`:

```
1 for i in range(10):  
2     print(it.__next__())  
3 print(it.__next__()) # Tratamos de imprimir un valor más
```

- Produce:

```
1 Traceback (most recent call last):  
2   File "/home/patxi/git/lp-python-2026/tema-6/ejem3-func-generadoras.py", 1  
3     print(it.__next__())  
4           ^^^^^^^^^^^^^^^  
5 StopIteration
```



# GESTIÓN DE EXCEPCIONES

- Python permite capturar los errores que puedan producirse durante la ejecución de nuestro código
- Estructura básica:

```
1 try:
2     numero = int(input("Introduce un número: "))
3     resultado = 10 / numero
4     print(f"Resultado: {resultado}")
5 except ValueError:
6     print("¡Error! Debes introducir un número válido.")
7 except ZeroDivisionError:
8     print("¡Error! No puedes dividir entre cero.")
```



# GESTIÓN DE EXCEPCIONES

- Si es necesario podemos capturar la variable de error:

```
1 try:
2     numero = int(input("Introduce un número: "))
3     resultado = 10 / numero
4     print(f"Resultado: {resultado}")
5 except ValueError e:
6     print("¡Error! Debes introducir un número válido.")
7     print(str(e))
8 except ZeroDivisionError e:
9     print("¡Error! No puedes dividir entre cero.")
```





# GESTIÓN DE EXCEPCIONES

- Podemos definir acciones a ejecutar si no hay errores en un bloque else:

```
1 try:
2     archivo = open("datos.txt", "r")
3 except FileNotFoundError:
4     print("El archivo no existe.")
5 else:
6     print("Archivo leído con éxito.")
7     print(archivo.read())
```



# GESTIÓN DE EXCEPCIONES

- Podemos también definir acciones que deben ejecutarse siempre:

```
1 try:
2     archivo = open("datos.txt", "r")
3 except FileNotFoundError:
4     print("El archivo no existe.")
5 else:
6     print("Archivo leído con éxito.")
7     print(archivo.read())
8 finally:
9     print("Cerrando recursos...")
10    # Esto asegura que si el archivo se abrió, se cierre pase lo que pase
11    if archivo != None and not archivo.closed:
12        archivo.close()
```



# GESTIÓN DE EXCEPCIONES

- Podemos lanzar excepciones desde nuestro código si queremos indicar una situación de error:

```
1 def establecer_edad(edad):  
2     if edad < 0:  
3         raise ValueError("La edad no puede ser negativa.")  
4     print(f"Edad establecida en {edad}")  
5  
6 try:  
7     establecer_edad(-5)  
8 except ValueError as e:  
9     print(f"Capturamos un error personalizado: {e}")
```



# MÁS SOBRE ARGUMENTOS POSICIONALES Y NOMBRADOS

- Una función en Python puede recibir más argumentos de los especificados en su definición
- Estos argumentos extra pueden ser capturados:
  - con la notación `*args` para parámetros posicionales (args se recibe como una lista)
  - con `**kwargs` para parámetros nombrados (kwargs se recibe como un diccionario)
- `*args` debe aparecer siempre antes de `**kwargs`
- Ejemplo 1:

```
1 def strange_func1(a, b, *args):
2     print(f"a: {a}")
3     print(f"b: {b}")
4     for arg in args: # args es una lista, posiblemente vacía
5         print(f"arg: {arg}")
6
7 strange_func1(1, 2, 3, 4, 5)
```



# MÁS SOBRE ARGUMENTOS POSICIONALES Y NOMBRADOS

- Ejemplo 2:

```
1 def strange_func2(a, b, **kwargs):  
2     print(f"a: {a}")  
3     print(f"b: {b}")  
4     for key, value in kwargs.items(): # kwargs es un diccionario  
5         print(f"{key}: {value}")  
6  
7 strange_func2(1, 2, c=3, d=4, e=5)
```



# MÁS SOBRE ARGUMENTOS POSICIONALES Y NOMBRADOS

- Ejemplo 3:

```
1 def strange_func3(a, b, *args, **kwargs):
2     print(f"a: {a}")
3     print(f"b: {b}")
4     for arg in args:
5         print(f"arg: {arg}")
6     for key, value in kwargs.items():
7         print(f"{key}: {value}")
8
9 strange_func3(1, 2, 3, 4, 5, c=6, d=7, e=8)
```



# MÁS SOBRE ARGUMENTOS POSICIONALES Y NOMBRADOS

- Si pasamos argumentos posicionales cuando se esperan argumentos nombrados o viceversa, obtenemos un error:

```
1 strange_func2(1, 2, 3, 4, 5)
```

- Produce:

```
1 TypeError: strange_func2() takes 2 positional arguments but 5 were given
```



# MÁS SOBRE ARGUMENTOS POSICIONALES Y NOMBRADOS

- Si tenemos una lista o un diccionario podemos pasar su contenido a una función:
  - Para listas usamos el operador `*` (argumentos posicionales)
  - Para diccionarios usamos el operador `**` (argumentos nombrados)
- Ejemplo:

```
1 numeros = [1, 2, 3, 4, 5]
2 cosas_nombradas = {"c": 6, "d": 7, "e": 8}
3
4 strange_func3(0, 0, *numeros, **cosas_nombradas)
5 # Es equivalente a pasar
6 strange_func3(0, 0, 1, 2, 3, 4, 5, c=6, d=7, e=8)
```

